

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу
«Операционные системы»

Группа: М8О-215Б-23

Студент: Аралов И.А.

Преподаватель: Миронов Е.С.

Оценка: _____

Дата: 04.03.24

Москва, 2024

Постановка задачи

Вариант 20.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Дан массив координат (x, y, z) . Необходимо найти три точки, которые образуют треугольник максимальной площади

Общий метод и алгоритм решения

Использованные системные вызовы:

- `malloc` — выделяет память в куче для динамических массивов и структур.
- `free` — освобождает память, ранее выделенную с помощью `malloc`.
- `pthread_create` — создает новый поток и запускает выполнение функции в этом потоке.
- `pthread_join` — ожидает завершения выполнения указанного потока.
- `pthread_exit` — завершает выполнение текущего потока.
- `gettimeofday` — получает текущее время с точностью до микросекунд.
- `srand` — инициализирует генератор случайных чисел.
- `rand` — генерирует псевдослучайное число.
- `time` — возвращает текущее время в секундах (используется для инициализации генератора случайных чисел).

Алгоритм работы программы

1. Инициализация программы:

- Программа принимает аргумент командной строки `maxThreads`, который определяет максимальное количество потоков для выполнения вычислений.
- Пользователь вводит размер массива точек (`arraySize`), который будет использоваться для генерации случайных координат точек.

2. Генерация массива точек:

- Программа выделяет память для массива точек (`arr`) размером `arraySize`.
- Каждая точка инициализируется случайными значениями координат x, y, z в диапазоне от 0 до 99.

3. Запуск многопоточной обработки:

- Программа создает `maxThreads` потоков, каждый из которых будет выполнять функцию `maxAreaThread`.
- Каждому потоку передается структура `ThreadData`, содержащая:
 - Указатель на массив точек (`array`).
 - Количество потоков (`n`).
 - Идентификатор потока (`thread`).
 - Размер массива (`arraysize`).

4. Вычисление максимальной площади в каждом потоке:

- В функции `maxAreaThread` каждый поток перебирает все возможные комбинации из трёх точек в массиве.
- Для каждой комбинации вычисляется площадь треугольника с помощью функции `triangleArea`.
- Если площадь текущего треугольника больше максимальной найденной площади, обновляются значения максимальной площади и соответствующих точек.
- Результаты работы каждого потока сохраняются в структуре `ExitData`, которая содержит:
 - Три точки, образующие треугольник.
 - Площадь этого треугольника.

5. Сбор результатов от всех потоков:

- Основной поток ожидает завершения всех созданных потоков с помощью `pthread_join`.
- Результаты каждого потока сохраняются в массиве `exit_array`.

6. Определение глобального максимума:

- Основной поток перебирает результаты всех потоков и находит комбинацию точек с максимальной площадью.
- Эта комбинация сохраняется в переменной `otvet`.

7. Вывод результатов:

- Программа выводит координаты трёх точек, образующих треугольник с максимальной площадью.
- Выводится значение максимальной площади.
- Выводится время выполнения вычислений.
- Выводится количество использованных потоков.

8. Освобождение ресурсов:

- Программа освобождает память, выделенную для массива точек и других структур данных.

Код программы

```
//main.c

#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>

#include <string.h>

#include <sys/time.h>

#include <time.h>

#include <math.h>


typedef struct {

    double x, y, z;

} Point;


typedef struct {

    Point *array;

    int n;

    int thread;

    int arraysize;

} ThreadData;


typedef struct {

    Point array[3];
```

```

    double area;

} ExitData;

double triangleArea(Point A, Point B, Point C) {

    const double epsilon = 1e-9; // Точность для проверки коллинеарности

    // Вычисляем векторы AB и AC

    Point AB = {B.x - A.x, B.y - A.y, B.z - A.z};

    Point AC = {C.x - A.x, C.y - A.y, C.z - A.z};

    // Вычисляем векторное произведение AB × AC

    Point cross;

    cross.x = AB.y * AC.z - AB.z * AC.y;

    cross.y = AB.z * AC.x - AB.x * AC.z;

    cross.z = AB.x * AC.y - AB.y * AC.x;

    // Вычисляем длину вектора произведения

    double length = sqrt(cross.x * cross.x + cross.y * cross.y + cross.z * cross.z);

    // Если точки коллинеарны (площадь нулевая)

    if (length < epsilon) {

        return 0.0;

    }

    return length / 2.0;

}

//сортировка maxArea, выполняемая в потоке

void *maxAreaThread(void *arg) {

    ThreadData *data = (ThreadData *)arg;

```

```

ExitData exit_data;

Point *arr = data -> array;

int step = data -> n;

int i = data -> thread;

int size = data -> arraysize;


double max = 0;

for (i; i < size - 2; i += step){

    for (int j = i + 1; j < size - 1; j++){

        for (int k = j + 1; k < size; k++){

            {

                if (triangleArea(arr[i], arr[j], arr[k]) > max){

                    exit_data.area = triangleArea(arr[i], arr[j], arr[k]);

                    exit_data.array[0] = arr[i];

                    exit_data.array[1] = arr[j];

                    exit_data.array[2] = arr[k];

                }

            }

        }

    }

}

ExitData *exit_data_ptr = malloc(sizeof(ExitData));

*exit_data_ptr = exit_data;

pthread_exit((void*)exit_data_ptr);

}

//основная ф-я maxArea

ExitData maxArea(Point arr[], int n, int maxThreads) {

```

```
pthread_t *threads = (pthread_t *)malloc(maxThreads * sizeof(pthread_t));

ThreadData *threadData = (ThreadData *)malloc(maxThreads * sizeof(ThreadData));

for (int i = 0; i < maxThreads; i++) {

    threadData[i].array = arr;

    threadData[i].n = maxThreads;

    threadData[i].thread = i;

    threadData[i].arraysize = n;

    pthread_create(&threads[i], NULL, maxAreaThread, (void *)&threadData[i]);

}

// Массив для хранения результатов

ExitData *exit_array = (ExitData *)malloc(maxThreads * sizeof(ExitData));

// Ожидаем завершения потоков и сохраняем результаты

for (int i = 0; i < maxThreads; i++) {

    void *exit_data;

    pthread_join(threads[i], &exit_data); // Получаем указатель на ExitData

    exit_array[i] = *(ExitData *)exit_data; // Копируем данные

    free(exit_data); // Освобождаем память, выделенную в потоке

}

double result = 0;

int index = -1;

for (int i = 0; i < maxThreads; i++)

{

    if (exit_array[i].area > result){

        result = exit_array[i].area;

    }

}
```

```

        index = i;

    }

}

free(threads);

free(threadData);

return exit_array[index];
}

int main(int argc, char *argv[]) {

    if (argc < 2) {

        fprintf(stderr, "Usage: %s <max_threads>\n", argv[0]);

        return 1;

    }

    int maxThreads = atoi(argv[1]);

    int arraySize;

    printf("Enter the number of elements in the array: ");

    scanf("%d", &arraySize);

    Point *arr = (Point *)malloc(arraySize * sizeof(Point));

    // Инициализация генератора случайных чисел

    srand(time(NULL));

    // Генерация случайных чисел для массива

```



```
for (int i = 0; i < arraySize; i++) {

    arr[i].x = rand() % 100; // Генерация случайных чисел от 0 до 99

    arr[i].y = rand() % 100;

    arr[i].z = rand() % 100;

}


// printf("Original array:\n");

// for (int i = 0; i < arraySize; i++) {

//     printf("{%.2f, %.2f, %.2f}\n", arr[i].x, arr[i].y, arr[i].z);

// }

// printf("\n");


struct timeval start, end;

gettimeofday(&start, NULL);


ExitData otvet = maxArea(arr, arraySize, maxThreads);


gettimeofday(&end, NULL);


for (int i = 0; i < 3; i++)

{

    printf("{%.2f, %.2f, %.2f} - Point %d\n", otvet.array[i].x, otvet.array[i].y,
otvet.array[i].z, i);


}printf("%f - Max area\n", otvet.area);


long seconds = end.tv_sec - start.tv_sec;
```

```
long microseconds = end.tv_usec - start.tv_usec;

double elapsed = seconds + microseconds * 1e-6;

printf("Time taken for calculating: %.6f seconds\n", elapsed);

printf("Number of threads used: %d\n", maxThreads);

free(arr);

return 0;
}
```

```
read(3, "\\177ELF\\2\\1\\1\\3\\0\\0\\0\\0\\0\\0\\0\\0\\3\\0>\\0\\1\\0\\0\\0\\0\\0\\0\\0\\0\\0\\0\\0"..., 832) = 832
```

```

newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=907784, ...}, AT_EMPTY_PATH) = 0

mmap(NULL, 909560, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fae2cd58000

mmap(0x7fae2cd68000, 471040, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x10000) = 0x7fae2cd68000

mmap(0x7fae2cddb000, 368640, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x83000) = 0x7fae2cddb000

mmap(0x7fae2ce35000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0xdc000) = 0x7fae2ce35000

close(3) = 0

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\20t\2\0\0\0\0"... , 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64)
= 784

newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1922136, ...}, AT_EMPTY_PATH) = 0

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64)
= 784

mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fae2cb77000

mmap(0x7fae2cb9d000, 1396736, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x26000) = 0x7fae2cb9d000

mmap(0x7fae2ccf2000, 339968, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x17b000) = 0x7fae2ccf2000

mmap(0x7fae2cd45000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1ce000) = 0x7fae2cd45000

mmap(0x7fae2cd4b000, 53072, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,
-1, 0) = 0x7fae2cd4b000

close(3) = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fae2cb74000

arch_prctl(ARCH_SET_FS, 0x7fae2cb74740) = 0

set_tid_address(0x7fae2cb74a10) = 88342

set_robust_list(0x7fae2cb74a20, 24) = 0

rseq(0x7fae2cb75060, 0x20, 0, 0x53053053) = 0

mprotect(0x7fae2cd45000, 16384, PROT_READ) = 0

mprotect(0x7fae2ce35000, 4096, PROT_READ) = 0

mprotect(0x403000, 4096, PROT_READ) = 0

mprotect(0x7fae2ce70000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

```

```

munmap(0x7fae2ce37000, 25258) = 0

newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH)
= 0

getrandom("\xcf\x50\x7f\x0b\x89\x8a\xf5\xec", 8, GRND_NONBLOCK) = 8

brk(NULL) = 0x1448000

brk(0x1469000) = 0x1469000

newfstatat(0, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH)
= 0

write(1, "Enter the number of elements in "..., 43Enter the number of elements in the
array: ) = 43

read(0, 1000

"1000\n", 1024) = 5

rt_sigaction(SIGRT_1, {sa_handler=0x7fae2cbfd6a0, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7fae2cbb3050}, NULL, 8)
= 0

rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
0x7fae2c373000

mprotect(0x7fae2c374000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CL
ONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2cb73990,
parent_tid=0x7fae2cb73990, exit_signal=0, stack=0x7fae2c373000, stack_size=0x7fff80,
tls=0x7fae2cb736c0}strace: Process 88370 attached

=> {parent_tid=[88370]}, 88) = 88370

[pid 88370] rseq(0x7fae2cb73fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88370] <... rseq resumed>) = 0

[pid 88342] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88370] set_robust_list(0x7fae2cb739a0, 24 <unfinished ...>

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>

[pid 88370] <... set_robust_list resumed>) = 0

[pid 88342] <... mmap resumed>) = 0x7fae2bb72000

[pid 88370] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88342] mprotect(0x7fae2bb73000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

[pid 88370] <... rt_sigprocmask resumed>NULL, 8) = 0

```

```

[pid 88342] <... mprotect resumed>)      = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2c372990,
parent_tid=0x7fae2c372990, exit_signal=0, stack=0x7fae2bb72000, stack_size=0x7fff80,
tls=0x7fae2c3726c0}strace: Process 88371 attached

=> {parent_tid=[88371]}, 88) = 88371

[pid 88371] rseq(0x7fae2c372fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88371] <... rseq resumed>)          = 0

[pid 88342] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88371] set_robust_list(0x7fae2c3729a0, 24 <unfinished ...>

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>

[pid 88371] <... set_robust_list resumed>) = 0

[pid 88342] <... mmap resumed>)          = 0x7fae2b371000

[pid 88371] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88342] mprotect(0x7fae2b372000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

[pid 88371] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88342] <... mprotect resumed>)      = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2bb71990,
parent_tid=0x7fae2bb71990, exit_signal=0, stack=0x7fae2b371000, stack_size=0x7fff80,
tls=0x7fae2bb716c0}strace: Process 88372 attached

<unfinished ...>

[pid 88372] rseq(0x7fae2bb71fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88342] <... clone3 resumed> => {parent_tid=[88372]}, 88) = 88372

[pid 88372] <... rseq resumed>)          = 0

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88372] set_robust_list(0x7fae2bb719a0, 24 <unfinished ...>

[pid 88342] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88372] <... set_robust_list resumed>) = 0

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>

```

```

[pid 88372] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88342] <... mmap resumed>)          = 0x7fae2ab70000

[pid 88372] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88342] mprotect(0x7fae2ab71000, 8388608, PROT_READ|PROT_WRITE) = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2b370990,
parent_tid=0x7fae2b370990, exit_signal=0, stack=0x7fae2ab70000, stack_size=0x7fff80,
tls=0x7fae2b3706c0}strace: Process 88373 attached

=> {parent_tid=[88373]}, 88) = 88373

[pid 88373] rseq(0x7fae2b370fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88373] <... rseq resumed>)          = 0

[pid 88342] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88373] set_robust_list(0x7fae2b3709a0, 24 <unfinished ...>

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>

[pid 88373] <... set_robust_list resumed>) = 0

[pid 88342] <... mmap resumed>)          = 0x7fae2a36f000

[pid 88373] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88342] mprotect(0x7fae2a370000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

[pid 88373] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88342] <... mprotect resumed>)      = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2ab6f990,
parent_tid=0x7fae2ab6f990, exit_signal=0, stack=0x7fae2a36f000, stack_size=0x7fff80,
tls=0x7fae2ab6f6c0}strace: Process 88374 attached

=> {parent_tid=[88374]}, 88) = 88374

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88374] rseq(0x7fae2ab6ffe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88342] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88374] <... rseq resumed>)          = 0

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>

```

```

[pid 88374] set_robust_list(0x7fae2ab6f9a0, 24 <unfinished ...>

[pid 88342] <... mmap resumed>) = 0x7fae29b6e000

[pid 88342] mprotect(0x7fae29b6f000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

[pid 88374] <... set_robust_list resumed>) = 0

[pid 88342] <... mprotect resumed>) = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>

[pid 88374] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88342] <... rt_sigprocmask resumed>[], 8) = 0

[pid 88374] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2a36e990,
parent_tid=0x7fae2a36e990, exit_signal=0, stack=0x7fae29b6e000, stack_size=0x7fff80,
tls=0x7fae2a36e6c0} => {parent_tid=[88375]}, 88) = 88375

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0)
= 0x7fae2936d000

[pid 88342] mprotect(0x7fae2936e000, 8388608, PROT_READ|PROT_WRITE) = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae29b6d990,
parent_tid=0x7fae29b6d990, exit_signal=0, stack=0x7fae2936d000, stack_size=0x7fff80,
tls=0x7fae29b6d6c0} => {parent_tid=[88376]}, 88) = 88376

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

[pid 88342] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0)
= 0x7fae28b6c000

[pid 88342] mprotect(0x7fae28b6d000, 8388608, PROT_READ|PROT_WRITE) = 0

[pid 88342] rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

[pid 88342]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_S
ETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fae2936c990,
parent_tid=0x7fae2936c990, exit_signal=0, stack=0x7fae28b6c000, stack_size=0x7fff80,
tls=0x7fae2936c6c0}strace: Process 88377 attached

=> {parent_tid=[88377]}, 88) = 88377

[pid 88377] rseq(0x7fae2936cfe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88342] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88377] <... rseq resumed>) = 0

[pid 88342] <... rt_sigprocmask resumed>NULL, 8) = 0

```

```

[pid 88377] set_robust_list(0x7fae2936c9a0, 24 <unfinished ...>

[pid 88342] futex(0x7fae2cb73990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 88370, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 88377] <... set_robust_list resumed>) = 0

[pid 88377] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

strace: Process 88376 attached

strace: Process 88375 attached

[pid 88376] rseq(0x7fae29b6dfe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88375] rseq(0x7fae2a36efe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 88376] <... rseq resumed>          = 0

[pid 88375] <... rseq resumed>          = 0

[pid 88376] set_robust_list(0x7fae29b6d9a0, 24 <unfinished ...>

[pid 88375] set_robust_list(0x7fae2a36e9a0, 24 <unfinished ...>

[pid 88376] <... set_robust_list resumed>) = 0

[pid 88375] <... set_robust_list resumed>) = 0

[pid 88376] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88375] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 88376] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88375] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 88376] mmap(NULL, 134217728, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_NORESERVE,
-1, 0) = 0x7fae20b6c000

[pid 88376] munmap(0x7fae20b6c000, 55132160) = 0

[pid 88376] munmap(0x7fae28000000, 11976704) = 0

[pid 88376] mprotect(0x7fae24000000, 135168, PROT_READ|PROT_WRITE) = 0

[pid 88376] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

[pid 88376] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=25258, ...},
AT_EMPTY_PATH) = 0

[pid 88376] mmap(NULL, 25258, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fae2ce37000

[pid 88376] close(3)                    = 0

[pid 88376] openat(AT_FDCWD, "/usr/local/lib64/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3

[pid 88376] read(3,
"\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832

[pid 88376] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=906528, ...},
AT_EMPTY_PATH) = 0

```



```

[pid 88376] mmap(NULL, 181160, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7fae28b3f000

[pid 88376] mmap(0x7fae28b43000, 143360, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x7fae28b43000

[pid 88376] mmap(0x7fae28b66000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x27000) = 0x7fae28b66000

[pid 88376] mmap(0x7fae28b6a000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2b000) = 0x7fae28b6a000

[pid 88376] close(3) = 0

[pid 88376] mprotect(0x7fae28b6a000, 4096, PROT_READ) = 0

[pid 88376] munmap(0x7fae2ce37000, 25258) = 0

[pid 88376] futex(0x7fae28b6b230, FUTEX_WAKE_PRIVATE, 2147483647) = 0

[pid 88376] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 88376] madvise(0x7fae2936d000, 8368128, MADV_DONTNEED) = 0

[pid 88376] exit(0) = ?

[pid 88376] +++ exited with 0 +++

[pid 88371] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 88371] madvise(0x7fae2bb72000, 8368128, MADV_DONTNEED) = 0

[pid 88371] exit(0) = ?

[pid 88371] +++ exited with 0 +++

[pid 88375] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 88375] madvise(0x7fae29b6e000, 8368128, MADV_DONTNEED) = 0

[pid 88375] exit(0) = ?

[pid 88375] +++ exited with 0 +++

[pid 88377] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 88377] madvise(0x7fae28b6c000, 8368128, MADV_DONTNEED) = 0

[pid 88377] exit(0) = ?

[pid 88377] +++ exited with 0 +++

[pid 88372] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 88372] madvise(0x7fae2b371000, 8368128, MADV_DONTNEED) = 0

[pid 88372] exit(0) = ?

[pid 88372] +++ exited with 0 +++

[pid 88374] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 88374] madvise(0x7fae2a36f000, 8368128, MADV_DONTNEED) = 0

[pid 88374] exit(0) = ?

```

[illegible]

Наблюдения

Массив из 1000 элементов

Число потоков	Время исполнения (с)	Ускорение	Эффективность
1	5.293175	1	1
2	3.277967	1.61	0.81
3	2.606157	2.04	0.68
4	2.213601	2.4	0.6
5	2.023004	2.62	0.52
6	1.828773	2.91	0.49
7	1.690052	3.13	0.45

Ускорение показывает во сколько раз применение параллельного алгоритма уменьшает время решения задачи по сравнению с последовательным алгоритмом. Ускорение определяется величиной $S_N = T_1 / T_N$, где T_1 - время выполнения на одном потоке, T_N - время выполнения на N потоках.

Эффективность - величина $E_N = S_N / N$, где S_N - ускорение, N - количество используемых потоков.

Вывод

На основе тестирования программы с разным количеством потоков и объемом данных можно сделать следующие выводы:

1. Многопоточность значительно ускоряет выполнение программы при грамотном распределении нагрузки между потоками.
 2. Для максимального ускорения следует выбирать оптимальное количество потоков, которое соответствует вычислительным возможностям компьютера (например, числу ядер процессора) и объему задачи.
 3. Избыточное количество потоков может снижать эффективность работы из-за накладных расходов на управление потоками и синхронизацию.
- Таким образом, многопоточность является эффективным инструментом для повышения производительности, если её правильно применять в зависимости от аппаратных характеристик и сложности задачи.